

1.

The Shift to Chiplet Hardware

Older generation GPUs, like the MI200, were built using a "monolithic" design where all computing power lived on one massive die. The newer MI300 architecture breaks this apart into a disaggregated, chiplet-based design featuring multiple smaller compute units called XCDs .

While this chiplet approach is highly scalable, it introduces a physical challenge: Non-Uniform Memory Access (NUMA). Because compute and memory are now spread out, accessing "remote" memory on a different part of the GPU can account for up to 20 percent of a task's total latency .

The Software Bottleneck

Traditional GPU programming struggles to adapt to this new hardware efficiently. Usually, a host CPU launches a tiny kernel of work, waits for it to finish, and then launches the next one. This constant starting and stopping creates idle "bubbles" where the GPU sits doing nothing.

The Solution: Persistent Kernels and MLIR-AIR

To keep the GPU constantly fed with work, AMD proposes using "Persistent Kernels" (or Megakernels). Instead of the CPU micromanaging every task, one single, continuous kernel is launched on the GPU. This kernel acts as a tiny on-device task scheduler, overlapping memory and compute tasks to eliminate idle bubbles.

However, for a persistent kernel to know exactly what to do, it needs a perfect blueprint. This is provided by the MLIR-AIR Compiler. The compiler explicitly manages how compute and memory are placed across the physical chiplets . It does this in four distinct stages:

1. **Operator Level:** It takes massive machine learning operators and tiles them.
2. **Task Graph Level:** It analyzes the exact dependencies between these tiles to figure out the most efficient order of operations .
3. **Spatial Execution Plan:** It assigns these specific tasks to specific physical XCDs and compute units to minimize the penalty of remote memory access .
4. **GPU Shader Code:** It translates this spatial plan into low-level code that the persistent kernel can run.

Results and Future Outlook

This spatial, persistent approach shows real promise. When tested using an inference system called Mirage on the MI300, the megakernel approach yielded speedups of up to 17 percent in memory-bound scenarios compared to standard frameworks like vLLM . Ultimately, to get the most out of AMD's modern chiplet based GPUs, compilers must evolve to explicitly manage the placement and synchronization of tasks across time and space .

2.

- Create MLIR dialect/toolchain that converts Triton matmult kernel to AWS Trainium [matmul](#) kernel
- Create MLIR dialect/toolchain that converts Triton vector add kernel to AWS Trainium vector add kernel using [tensor_tensor](#)
- Try and optimize the matmult and vector add kernel on Trainium
- Plan
 - Lower Triton to Triton IR
 - Lower Triton IR to Linalg
 - Lower Linalg generic to specialized Linalg operator ([matmul](#), [add](#))
 - Create dialect to raise from Linalg dialect to NNI python source code